SIMATS ENGINEERING

ASSESSMENT TOOL 2 – COMPLETE ANSWER

Course: Artificial Intelligence | Course Code: CSA17

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and models for classification and decision-making.

Student Name: _____ **Reg. No:** _____

Duration: 1 Hour **Total Marks:** 40

Industry Scenario: A healthcare company wants to predict early diabetes diagnosis from patient records and train an AI agent to recommend treatment actions based on patient-state transitions.

Important: The code uses a synthetic dataset generated inside Python so the complete answer can run without needing a private patient file. In an actual clinical project, the same pipeline should be trained and validated on approved, de-identified patient data.

TASK 1: DATA PREPARATION & INDUCTIVE LEARNING

1(a) Inductive learning approach, target and key features

Inductive learning learns a general prediction rule from labelled examples and applies it to unseen patients. For this problem, supervised classification is suitable because historical patient records have a known diabetes outcome. A practical pipeline is: data validation → missing-value handling → encoding → train/test split → imbalance handling on training data only → model training → evaluation.

Target variable: Diabetes (0 = non-diabetic, 1 = diabetic).

Key features: Glucose level is expected to be highly predictive because elevated glucose is directly associated with diabetes; BMI captures obesity-related metabolic risk; age reflects increasing diabetes risk with age; blood pressure and cholesterol provide additional cardiovascular/metabolic risk information; family history captures inherited predisposition.

1(b) Handling class imbalance

Recommended strategy: SMOTE on the training set only. If diabetic cases are the minority, simply optimizing accuracy can produce a model that predicts most patients as non-diabetic. SMOTE creates synthetic minority-class training examples, helping the classifier learn the diabetic class. It must be applied after the train/test split so synthetic information cannot leak into the test set. Recall, precision, F1 and ROC-AUC should be monitored in addition to accuracy.

Class weights are also a strong alternative, particularly for clinical deployment, because they do not synthetically alter patient observations. The final choice should be validated using the actual dataset and clinical objectives.

1(c) 70:30 split and preprocessing

70:30 split: 70% provides enough labelled cases for model learning while reserving 30% for a reasonably large independent evaluation set. For medical data, use stratification so both diabetic and non-diabetic classes remain represented in similar proportions. In a production study, an external validation cohort or temporal split is preferable when available.

Preprocessing: numeric missing values → median imputation; categorical values → most-frequent imputation and one-hot encoding; numeric scaling → StandardScaler for Logistic Regression; Decision Trees do not require normalization. Scaling prevents large numerical ranges from dominating distance/gradient-based statistical models, while tree split decisions are generally invariant to monotonic scaling.

Complete Python implementation used for Tasks 1–3

```
# CSA17 Assessment Tool 2
# Healthcare diabetes prediction: Decision Tree + Logistic Regression
# + SMOTE + pruning + feature importance

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, classification_report,
    roc_curve
)
from sklearn.inspection import permutation_importance
```

```
# If imblearn is not installed:
# pip install imbalanced-learn
from imblearn.over_sampling import SMOTE

# ------------------------------------------------------------
# 1. Create a reproducible synthetic healthcare dataset
# ------------------------------------------------------------
rng = np.random.default_rng(42)
n = 600

age = rng.integers(20, 81, n)
bp = np.clip(rng.normal(125 + 0.15*(age-40), 15, n), 85, 190)
chol = np.clip(rng.normal(190 + 0.35*(age-40), 35, n), 100, 320)
glucose = np.clip(rng.normal(105 + 0.35*(age-40), 30, n), 55, 250)
bmi = np.clip(rng.normal(27, 5, n), 16, 45)
family = rng.binomial(1, 0.30, n)

# Diabetes probability: deliberately imbalanced
risk = (
    -7.0
    + 0.035*age
    + 0.018*(bp-120)
    + 0.006*(chol-180)
    + 0.045*(glucose-100)
    + 0.10*(bmi-25)
    + 0.80*family
)
prob = 1 / (1 + np.exp(-risk))
diabetes = rng.binomial(1, prob)

df = pd.DataFrame({
    "Age": age,
    "BloodPressure": bp,
    "Cholesterol": chol,
    "Glucose": glucose,
    "BMI": bmi,
    "FamilyHistory": family,
    "Diabetes": diabetes
})

# Introduce a few missing numeric values for preprocessing demo
for col in ["BloodPressure", "Cholesterol", "BMI"]:
    idx = rng.choice(n, size=6, replace=False)
    df.loc[idx, col] = np.nan

print("First 5 rows:")
print(df.head())
print("\nData types:")
print(df.dtypes)
print("\nClass distribution:")
print(df["Diabetes"].value_counts())

X = df.drop(columns="Diabetes")
y = df["Diabetes"]

# 70:30 stratified split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=42
)

# ------------------------------------------------------------
# 2. SMOTE: impute first, then balance TRAINING data only
# ------------------------------------------------------------
imputer = SimpleImputer(strategy="median")
X_train_imp = pd.DataFrame(
    imputer.fit_transform(X_train),
    columns=X_train.columns, index=X_train.index
)
X_test_imp = pd.DataFrame(
    imputer.transform(X_test),
    columns=X_test.columns, index=X_test.index
)

smote = SMOTE(random_state=42)
X_train_bal, y_train_bal = smote.fit_resample(X_train_imp, y_train)

print("\nBefore SMOTE:")
print(y_train.value_counts())
print("After SMOTE:")
print(pd.Series(y_train_bal).value_counts())

# ------------------------------------------------------------
# 3. Decision Tree: pre-pruning
# ------------------------------------------------------------
dt_pre = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    min_samples_leaf=8,
    class_weight=None,
    random_state=42
)
dt_pre.fit(X_train_bal, y_train_bal)

pred_pre = dt_pre.predict(X_test_imp)
proba_pre = dt_pre.predict_proba(X_test_imp)[:, 1]

def metrics(name, y_true, pred, proba):
    print("\n", name)
```

```
    print("Accuracy :", accuracy_score(y_true, pred))
    print("Precision:", precision_score(y_true, pred, zero_division=0))
    print("Recall   :", recall_score(y_true, pred, zero_division=0))
    print("F1       :", f1_score(y_true, pred, zero_division=0))
    print("ROC-AUC  :", roc_auc_score(y_true, proba))
    print("Confusion matrix:\n", confusion_matrix(y_true, pred))
    print(classification_report(y_true, pred, zero_division=0))

metrics("Pre-pruned Decision Tree", y_test, pred_pre, proba_pre)

# Draw first three levels
plt.figure(figsize=(14, 8))
plot_tree(
    dt_pre,
    feature_names=X.columns,
    class_names=["Non-diabetic", "Diabetic"],
    filled=True,
    max_depth=2,
    rounded=True
)
plt.title("Decision Tree – First Three Levels")
plt.tight_layout()
plt.show()

# Root feature and Gini information
root_index = dt_pre.tree_.feature[0]
root_feature = X.columns[root_index]
print("\nRoot feature:", root_feature)
print("Root threshold:", dt_pre.tree_.threshold[0])
print("Root node Gini impurity:", dt_pre.tree_.impurity[0])

# ------------------------------------------------------------
# 4. Post-pruning using cost-complexity pruning
# ------------------------------------------------------------
path = DecisionTreeClassifier(
    criterion="gini", random_state=42
).cost_complexity_pruning_path(X_train_bal, y_train_bal)

best_model = None
best_score = -1
best_alpha = None

for alpha in path.ccp_alphas:
    tree = DecisionTreeClassifier(
        criterion="gini",
        ccp_alpha=alpha,
        random_state=42
    )
    tree.fit(X_train_bal, y_train_bal)
    pred = tree.predict(X_test_imp)
    score = f1_score(y_test, pred, zero_division=0)
    if score > best_score:
        best_score = score
        best_model = tree
        best_alpha = alpha

dt_post = best_model
pred_post = dt_post.predict(X_test_imp)
proba_post = dt_post.predict_proba(X_test_imp)[:, 1]

metrics("Post-pruned Decision Tree", y_test, pred_post, proba_post)
print("Selected ccp_alpha:", best_alpha)

print("\nComparison:")
print("Pre-pruning accuracy :", accuracy_score(y_test, pred_pre))
print("Post-pruning accuracy:", accuracy_score(y_test, pred_post))

# False Negatives
cm = confusion_matrix(y_test, pred_post)
tn, fp, fn, tp = cm.ravel()
print("\nFalse Negatives:", fn)
print("False Positives:", fp)

# ------------------------------------------------------------
# 5. Logistic Regression statistical learning model
# ------------------------------------------------------------
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_bal)
X_test_scaled = scaler.transform(X_test_imp)

lr = LogisticRegression(max_iter=2000, random_state=42)
lr.fit(X_train_scaled, y_train_bal)

pred_lr = lr.predict(X_test_scaled)
proba_lr = lr.predict_proba(X_test_scaled)[:, 1]

metrics("Logistic Regression", y_test, pred_lr, proba_lr)

# ------------------------------------------------------------
# 6. Feature importance
# ------------------------------------------------------------
dt_imp = pd.Series(
    dt_post.feature_importances_, index=X.columns
).sort_values(ascending=False)

lr_imp = pd.Series(
    np.abs(lr.coef_[0]), index=X.columns
).sort_values(ascending=False)
```

```
print("\nTop 3 Decision Tree predictors:")
print(dt_imp.head(3))

print("\nTop 3 Logistic Regression predictors:")
print(lr_imp.head(3))

# ROC curves
fpr1, tpr1, _ = roc_curve(y_test, proba_post)
fpr2, tpr2, _ = roc_curve(y_test, proba_lr)

plt.figure(figsize=(7, 5))
plt.plot(fpr1, tpr1, label="Decision Tree")
plt.plot(fpr2, tpr2, label="Logistic Regression")
plt.plot([0, 1], [0, 1], "--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC-AUC Comparison")
plt.legend()
plt.grid(True)
plt.show()
```

TASK 2: DECISION TREE FOR DIAGNOSIS

2(a) First three levels and root-node justification

Run the supplied program to display the first three levels using plot_tree(max_depth=2). The root feature is printed from the trained model. The root is selected because its split produces the greatest reduction in Gini impurity among candidate features/splits. In a typical diabetes dataset, **Glucose** is expected to be one of the strongest root candidates, but the exact root must be taken from the actual trained dataset output rather than assumed.

Gini impurity: Gini = 1 – Σp■². A pure node has Gini = 0. The chosen split minimizes weighted child-node impurity, equivalently maximizing impurity reduction.

2(b) Evaluation and False Negatives

The program reports Accuracy, Precision, Recall, F1-score, ROC-AUC and the confusion matrix. The confusion matrix is [[TN, FP], [FN, TP]]. A **False Negative** is a diabetic patient predicted as non-diabetic. In healthcare this is especially important because a missed diagnosis can delay testing, monitoring and intervention.

Ways to reduce False Negatives: prioritize recall/sensitivity during model selection; use class weights or SMOTE; adjust the probability decision threshold below 0.50 when clinically validated; collect more representative diabetic cases; and use a second confirmatory clinical test for borderline predictions. The threshold should be selected using validation data and clinician-approved operating requirements rather than arbitrarily.

2(c) Pre-pruning vs post-pruning

Pre-pruning limits tree complexity during training using parameters such as max_depth and min_samples_leaf. Post-pruning first allows a larger tree and then removes branches using cost-complexity pruning. The supplied code compares their test accuracy and also reports recall/F1. A clinically appropriate model should not be chosen from accuracy alone: sensitivity, calibration, AUC, stability, interpretability and clinical consequences of false negatives must also be considered.

Deployment recommendation: prefer the simpler, well-validated pruned model if its sensitivity and overall clinical performance remain acceptable. A slightly less accurate but stable and interpretable tree can be safer than an overfitted tree.

TASK 3: STATISTICAL LEARNING FOR RISK STRATIFICATION

3(a) Logistic Regression and comparison with Decision Tree

Logistic Regression models the probability of diabetes as a function of the predictors. Standardization is applied because the features have different numerical scales. The program reports Accuracy and ROC-AUC for both models and draws an ROC comparison plot.

When Logistic Regression is preferable: when a transparent relationship between predictors and outcome is desired, when probability estimates are important, when the relationship is approximately linear on the log-odds scale, and when interpretability is a major requirement. Coefficients can be inspected for direction and magnitude.

3(b) Feature importance – top 3 predictors

For the Decision Tree, importance is based on total reduction in impurity contributed by each feature. For Logistic Regression, the absolute standardized coefficient magnitude is used as a simple importance measure. The program prints the top three from each model.

Expected clinical pattern: Glucose is commonly among the strongest predictors, with BMI, age and family history also potentially important. The exact top three and their ranking should be copied from the program output. The two models may agree on major predictors but need not have identical rankings because they measure importance differently.

Interpretation: agreement between models increases confidence that the signal is robust, while disagreement can reveal nonlinear effects, feature interactions or correlated predictors. Feature importance shows association/predictive contribution, not causation.

TASK 4: REINFORCEMENT LEARNING FOR TREATMENT RECOMMENDATION

4(a) MDP formulation

States: simplified patient health stages such as S0 = high risk/poor control, S1 = moderate risk, S2 = improving/stable, S3 = controlled/healthy. A real system would use clinically validated state variables rather than only four labels.

Actions: Diet, Exercise, Medication, Monitor. Medication must not be autonomously prescribed by an RL system; in a real deployment it would require clinician oversight, contraindication checks and approved treatment protocols.

Reward: positive reward for clinically meaningful improvement, negative reward for deterioration, and a safety penalty for unsafe transitions. Example: +5 improvement, +1 stable, –5 deterioration, –10 unsafe action.

Transition dynamics: the next health state depends probabilistically on current state, selected action, patient characteristics, adherence and other factors. In real healthcare these probabilities must be learned/validated from safe clinical data and should not be fabricated as clinical truth.

4(b) Q-Learning – at least 5 episodes

```
# TASK 4: SAFE TOY MDP FOR EDUCATIONAL DEMONSTRATION
import numpy as np

states = ["S0_HighRisk", "S1_Moderate", "S2_Improving", "S3_Controlled"]
actions = ["Diet", "Exercise", "Medication", "Monitor"]

Q = np.zeros((4, 4))
alpha = 0.5
gamma = 0.9
epsilon = 0.10

# Educational toy transition model ONLY
# (not clinical guidance)
T = {
    (0,0):(1, 2), (0,1):(1, 3), (0,2):(1, 4), (0,3):(0, -1),
    (1,0):(2, 3), (1,1):(2, 4), (1,2):(2, 5), (1,3):(1, 1),
    (2,0):(3, 4), (2,1):(3, 5), (2,2):(3, 3), (2,3):(2, 1),
    (3,0):(3, 1), (3,1):(3, 1), (3,2):(3, 0), (3,3):(3, 2)
}

def choose_action(s):
    if np.random.random() < epsilon:
        return np.random.randint(4)
    return np.argmax(Q[s])

np.random.seed(42)

for ep in range(1, 6):
    s = 0
    for step in range(10):
        a = choose_action(s)
        ns, r = T[(s,a)]

        old_q = Q[s,a]
        Q[s,a] = old_q + alpha * (
            r + gamma*np.max(Q[ns]) - old_q
        )

        # Required Q-table updates can be recorded here
        if ep in [1, 2] and step == 0:
            print("Episode", ep,
                  "state-action:", states[s], actions[a],
                  "old Q:", round(old_q,3),
                  "new Q:", round(Q[s,a],3))

        s = ns
        if s == 3:
            break

print("\nFinal Q-table:")
print(np.round(Q, 3))

print("\nFinal policy:")
for s in range(4):
    print(states[s], "->", actions[np.argmax(Q[s])])

# Convergence criterion:
# stop when max absolute Q-table change over a complete episode
# is below 0.001 for 3 consecutive episodes in a real implementation.
```

Q-learning update equation: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max Q(s',a') - Q(s,a)]$. Here $\alpha = 0.5$ and $\gamma = 0.9$. The code prints actual updates for the first steps of Episodes 1 and 2, giving more than the required two iterations.

Illustrative first updates: If a transition from S0 using Diet gives reward +2 and the next-state maximum Q is initially 0, then $Q(S0,Diet) = 0 + 0.5[2 + 0.9(0) - 0] = 1.0$. If the same pair is later updated with reward +2 and the best next-state Q is 1.0, then $Q = 1.0 + 0.5[2 + 0.9(1) - 1.0] = 1.95$. These are educational MDP calculations, not medical treatment recommendations.

Convergence criterion: monitor the maximum absolute change in Q-values after each episode. A practical toy-environment criterion is $\max|Q_{new} - Q_{old}| < 0.001$ for three consecutive episodes. In a real medical RL system, convergence alone is not

sufficient; safety and policy validation are mandatory.

4(c) Final learned policy and ethics

Final policy: for each state, choose the action with the largest Q-value: $\pi(s) = \text{argmax}_a Q(s,a)$. The program prints the learned action for S0–S3 after five episodes.

Patient safety: treatment recommendations can cause physical harm. RL should operate under clinical rules, hard safety constraints and clinician supervision. Medication actions require medical authorization.

Bias and fairness: historical healthcare data can contain demographic and access-related biases. The policy must be evaluated across relevant patient groups, with fairness and subgroup performance checks.

Explainability: clinicians should be able to understand why a recommendation was produced. State variables, reward definitions, action constraints and policy reasoning should be auditable.

Privacy: patient records must be de-identified and handled under applicable healthcare privacy and security requirements. Data access should be minimized and audited.

Human oversight: an RL system should support—not replace—the clinician. A human must be able to reject or override a recommendation, especially when the patient's condition is uncertain or outside the model's validated operating range.

Validation: before deployment, evaluate retrospectively, prospectively and, where appropriate, in controlled clinical studies. Offline RL and constrained/safe RL are preferable starting points to unrestricted online experimentation with patients.

FINAL RESULTS / RECORD-WRITING FORMAT

Task	What to report
1	Target = Diabetes. Key features = Glucose, BMI, Age, Blood Pressure, Cholesterol, Family History. Use stratified 70:30 split; handle missing val
2	Decision Tree first three levels, root feature and Gini impurity, Accuracy, Precision, Recall, F1, confusion matrix, False Negatives, and pre-/post-
3	Logistic Regression Accuracy and ROC-AUC; Decision Tree vs Logistic Regression; top 3 predictors from both models; explain agreement/disag
4	MDP states/actions/rewards/transitions, Q-table updates, 5 episodes, convergence criterion, final policy and medical ethics.

Overall conclusion: The proposed system combines inductive supervised learning, Decision Tree diagnosis, statistical risk modelling and a constrained educational Q-Learning formulation. For a real healthcare product, model performance must be independently validated, patient safety must take priority over reward optimization, and treatment decisions must remain under qualified clinical oversight.